



HANGMAN GAME

Computer Organization & Assembly Language

Project Report

Group Members	Reg.id
Muhammad Hasnain	242259
Muhammad Bilal Khalid	243408
Muhammad Hissan	233119

Submitted to: Maam Seemab Khan

Table of Contents

	Catalog	
1. Introduction		3
2. Tools & Environment		4
3. Program Design		5
5. Key Assembly Language Concepts		8
6. Game Flow		10
7. Sample Program Output		10
8. Challenges Encountered		11
9. Learning Outcomes		11
10. Conclusion		11
Appendix: Assembly Glossary		13

1. Introduction

Hangman is a classic word-guessing game in which one player chooses a secret word and the other player attempts to reveal it by guessing individual letters, one at a time. For each incorrect guess, a body part is added to a stick-figure drawing until either the figure is complete (the player loses) or the word is fully revealed (the player wins).

This project implements Hangman entirely in x86 Assembly language using the MASM assembler and the Irvine32 library. The goal is to demonstrate a practical understanding of low-level programming concepts including register operations, memory addressing, conditional branching, procedure calls, and stack management — all core topics in the Computer Organization and Assembly Language (COAL) course.

1.1 Objectives

- Implement a fully playable console-based Hangman game in MASM x86 assembly.
- Use registers, memory segments (.data / .code), and the Irvine32 library correctly.
- Demonstrate procedure-based modular design with PROC / ENDP.
- Apply conditional jumps, loops, and stack operations for game logic.
- Provide a complete user experience with hints, tracked guesses, and a play-again option.

2. Tools & Environment

Tool / Component	Version / Details	Purpose
MASM	Visual Studio 2019/2022 (ml.exe)	Assembles .asm source into object code
Irvine32 Library	Kip Irvine – x86 Edition	Console I/O: WriteString, ReadChar, WriteDec, Clrscr, RandomRange
Visual Studio IDE	2019 / 2022	Assembling, linking, and debugging
Windows OS	32-bit subsystem	Target execution environment for Irvine32
x86 ISA	32-bit (IA-32)	CPU instruction set used throughout the project

MASM (Microsoft Macro Assembler): Translates human-readable assembly source files (.asm) into binary object files (.obj). The resulting object files are linked with the Irvine32 and Kernel32 libraries to produce a runnable Windows executable (.exe).

Irvine32 Library: Provides high-level wrapper procedures that simplify console operations. Key procedures used in this project include WriteString (print a null-terminated string), ReadChar (read a single keypress), WriteDec (print an unsigned integer), Clrscr (clear the console screen), Randomize (seed random generator), and RandomRange (return a random integer in a range).

3. Program Design

3.1 Memory Layout

The program uses two memory segments defined by MASM directives:

.data Segment: Contains all static data — secret words, hints, message strings, hangman ASCII art stages, and game-state variables. All strings are null-terminated byte arrays (BYTE ... ,0).

.code Segment: Contains all executable procedures. The main PROC is the entry point; all other procedures are called from it or from each other.

3.2 Data Segment Variables

Variable	Type	Description
word0 – word7	BYTE array	Eight null-terminated secret words (ASSEMBLY, COMPUTER, PROGRAM, KEYBOARD, MONITOR, NETWORK, IRVINE, WINDOWS)
wordTable	DWORD array	8-element pointer table — each DWORD holds the offset (address) of the corresponding word string
hint0 – hint7	BYTE array	Hint strings corresponding to each secret word
hintTable	DWORD array	8-element pointer table for hint strings
currentWord	DWORD	Holds the address of the randomly chosen secret word for the current round
currentHint	DWORD	Holds the address of the corresponding hint string
wordLen	DWORD	Length (number of characters) of the chosen secret word
wrongCount	DWORD	Counter for incorrect guesses; starts at 0, max is 6
guessedMask	BYTE[26]	26-byte array indexed A–Z. A value of 1 means the letter has been guessed
displayWord	BYTE[20]	Shown to the player — starts as underscores, correct guesses replace them
hang0 – hang6	BYTE array	Seven ASCII art strings representing the gallows from empty to fully drawn
hangTable	DWORD array	7-element pointer table for quick indexed access to gallows stages

4. Procedures (Code Segment)

The program is divided into eight procedures, each with a single clear responsibility:

4.1 main

Entry point of the program. Calls `Clrscr` to clear the screen, then enters the `@newGame` loop which calls `ResetGame`. The `@gameLoop` repeatedly draws the game state (hangman, word, hint, guessed letters), reads a letter, and processes the guess. Depending on the return value in `EAX` (0 = continue, 1 = win, 2 = lose), it jumps to the appropriate outcome screen. After a win or loss, it prompts the player to play again.

4.2 ResetGame

Zeros `wrongCount` and clears all 26 bytes of `guessedMask` to 0 using a loop. Then calls `ChooseWord` to select a new secret word and reset `displayWord`.

4.3 ChooseWord

Calls `Randomize` to seed the random number generator, then calls `RandomRange(8)` to obtain a random index 0–7 in `EAX`. Loads the chosen word address by indexing `wordTable` with `[esi + eax*4]`. Counts the word length by scanning until the null terminator is found. Fills `displayWord` with underscores equal to the word length. Finally, indexes `hintTable` with the same `EAX` to load the corresponding hint address.

4.4 GetUpperLetter

Prints the guess prompt, calls `ReadChar` to read one keypress into `AL`, then echoes it with `WriteChar`. If the character is lowercase ('a' – 'z'), it subtracts 32 to convert to uppercase (exploiting ASCII encoding). Validates the result is in range 'A'–'Z'; if not, loops back. Returns the valid uppercase letter in `AL`.

4.5 ProcessGuess (returns EAX: 0=continue, 1=win, 2=lose)

Converts the letter to a 0–25 index by subtracting 'A'. Checks `guessedMask[index]`; if already 1, prints a warning and returns 0. Otherwise marks the letter as guessed. Scans the secret word with a loop comparing each byte to the guessed letter and writing matches into `displayWord`. If no match was found, increments `wrongCount` and checks whether it has reached `MAX_WRONG` (6), returning 2 if so. If a match was found, calls `CheckWin` and returns 1 if won, 0 otherwise.

4.6 CheckWin

Loops through `displayWord` for `wordLen` iterations checking for any remaining underscore character. If an underscore is found, returns `EAX = 0` (not won). If the loop completes without finding one, returns `EAX = 1` (won).

4.7 DrawHangman

Copies wrongCount into EAX (capped at 6 to prevent out-of-bounds access). Indexes hangTable with `[esi + eax*4]` to obtain the address of the correct ASCII art string and calls WriteString to print it.

4.8 ShowDisplayWord

Iterates wordLen times, loading each character from displayWord with `movzx eax, BYTE PTR [esi]`, printing it with WriteChar, and printing a space after each character for readability.

4.9 ShowHint / ShowGuessedLetters

ShowHint prints the currentHint pointer via WriteString. ShowGuessedLetters loops A–Z (26 iterations), pushes outer loop registers, checks the inner displayWord for each guessed letter to determine whether it was correct or incorrect, and prints only incorrect guesses.

5. Key Assembly Language Concepts

5.1 Register Usage

Register	Role in this Program
EAX	General accumulator: holds word index, return values (0/1/2), integer for WriteDec, guessed letter
EBX	Holds the guessed letter (BL) during the search loop; outer letter counter in ShowGuessedLetters
ECX	Loop counter — wordLen for scanning, 26 for A–Z loop; automatically decremented by LOOP
EDX	Holds string offset passed to WriteString / WriteChar; also used as found-flag in ProcessGuess
ESI	Source pointer: points into secret word array or wordTable during reads
EDI	Destination pointer: points into displayWord when writing matched letters or initializing blanks

5.2 Pointer Arithmetic

The word, hint, and hangman pointer tables are arrays of 32-bit DWORD values (4 bytes each). To access element N, the program computes:

```
mov esi, OFFSET wordTable ; base address of table
mov edx, [esi + eax*4]    ; address = base + index × 4
```

This is equivalent to array indexing in high-level languages (e.g. wordTable[eax]) but expressed explicitly in assembly.

5.3 The LOOP Instruction

LOOP decrements ECX automatically and jumps back to the label if ECX $\neq$ 0. It is used in ChooseWord (count word length), ShowDisplayWord (print each character), and CheckWin (scan for underscores).

```
mov ecx, wordLen
@printLoop:
  movzx eax, BYTE PTR [esi]
  call WriteChar
  inc esi
  loop @printLoop ; ECX-- ; if ECX≠0 repeat
```


5.4 Conditional Branching

CMP subtracts the second operand from the first and updates CPU flags without storing the result. Conditional jump instructions (JE, JNE, JGE, JLE, JG) then test those flags:

```
cmp wrongCount, MAX_WRONG ; 6 wrong guesses?
jge @lose                ; if wrongCount >= 6, game over
```

5.5 Stack – PUSH / POP

In ShowGuessedLetters, the outer loop uses ECX, ESI, and EBX. Before the inner loop runs, these registers are pushed onto the stack and popped back after, preserving their values:

```
push ecx    ; save outer loop counter
push esi    ; save guessedMask pointer
push ebx    ; save current letter character
; ... inner loop ...
pop ebx
pop esi
pop ecx
```

5.6 ASCII Case Conversion

Lowercase ASCII letters 'a'–'z' have values 97–122. Uppercase 'A'–'Z' have values 65–90. The difference is exactly 32. Subtracting 32 converts any lowercase letter to uppercase:

```
cmp al, 'a'    ; al >= 'a'?
jl  @checkUpper
cmp al, 'z'    ; al <= 'z'?
jg  @checkUpper
sub al, 32     ; convert: 'a'(97) → 'A'(65)
```

5.7 MOVZX – Zero-Extension

When reading a single byte (BYTE PTR) from memory into a 32-bit register, the upper 24 bits could contain garbage. MOVZX (Move with Zero Extend) pads those bits with zeros, ensuring safe use of the full 32-bit register:

```
movzx eax, BYTE PTR [esi] ; safe: EAX = 0x000000xx
; vs
mov al, [esi]             ; risky: upper bits of EAX unchanged
```

6. Game Flow

The following steps describe a complete round:

1. Program starts → Clrscr → ResetGame (ChooseWord, clear arrays)
2. @gameLoop: Clrscr → DrawHangman → ShowDisplayWord → ShowHint → ShowGuessedLetters
3. GetUpperLetter: read keypress, convert to uppercase, validate A–Z
4. ProcessGuess: check duplicate → mark guessed → scan word → update wrongCount/displayWord
5. **EAX = 0:** continue → jump back to @gameLoop
6. **EAX = 1 (win):** render congratulations screen
7. **EAX = 2 (lose):** render game-over screen, reveal word
8. Play Again? Y → @newGame (repeat from step 1), any other key → WaitMsg + exit

7. Sample Program Output

Below is representative console output during a game session where the secret word is ASSEMBLY:

```
=====
```

HANGMAN GAME - IRVINE32

```
=====
```

```
+---+
```

```
|   |
```

```
|   |
```

```
|   |
```

```
|   |
```

```
|   |
```

```
===
```

Word: _ _ _ _ _ _ _

Hint: A popular operating system developed by Microsoft

Incorrect Guesses:

Enter a letter to guess (A-Z): |

8. Challenges Encountered

- **ShowGuessedLetters** uses an outer loop (A–Z, ECX = 26) and an inner loop (scan displayWord, ECX = wordLen). The inner loop would overwrite ECX, ESI, and EBX. This was solved by pushing them to the stack before the inner loop and popping them after. Register Preservation in Nested Loops:
- Accessing the correct string from wordTable required computing the byte offset as $\text{index} \times 4$ (since each DWORD pointer is 4 bytes). A mistake in this arithmetic would load a garbage address and crash the program. Pointer Table Indexing:
- **WriteString** prints null-terminated strings (EDX = address), while **WriteDec** prints an unsigned integer (EAX = value). Using the wrong procedure at any point causes either garbage output or a runtime error. Distinguishing WriteString from WriteDec:
- When **ProcessGuess** finds a match at position *i* in the secret word, it must write the letter to exactly position *i* in displayWord. Both ESI (secret word) and EDI (displayWord) are incremented in parallel during the search loop to maintain this alignment. displayWord Synchronisation:
- If wrongCount exceeded 6 due to an edge case, the program would index outside hangTable. A cap (`jle @validStage, else set eax = MAX_WRONG`) was added in **DrawHangman** to prevent this. Preventing Out-of-Bounds Hangman Stage:

9. Learning Outcomes

Completing this project provided hands-on experience with the following COAL concepts:

- Direct control of x86 CPU registers (EAX, EBX, ECX, EDX, ESI, EDI) for arithmetic, addressing, and I/O.
- Memory segmentation: understanding how .data and .code sections are physically organised.
- Null-terminated string management and byte-level memory access using pointer registers.
- Conditional branching logic (CMP + JE/JNE/JGE/JLE/JG) as the sole decision-making mechanism.
- The LOOP instruction as an efficient counted-iteration construct.
- PUSH/POP stack operations for register preservation across nested procedure calls.
- Pointer arithmetic for indexed access into DWORD tables (wordTable, hangTable).
- Modular procedure design (PROC/ENDP) with clearly defined register contracts and return values.

10. Conclusion

This project successfully implements a complete, interactive Hangman game in x86 Assembly using MASM and the Irvine32 library. Every aspect of the game — random word selection, letter tracking, ASCII art rendering, hint display, win/lose detection, and play-again logic — is implemented without

any high-level language constructs, relying entirely on registers, memory, conditional jumps, and procedures.

The project demonstrates that even complex interactive programs can be built at the assembly level with careful planning, modular procedure design, and a thorough understanding of CPU registers and memory addressing. The experience reinforces the COAL course objectives of understanding how high-level logic translates to machine-level instructions.

Appendix: Assembly Glossary

Instruction	Meaning
mov dst, src	Copy value from src into dst
cmp a, b	Subtract b from a; update CPU flags (result discarded)
je label	Jump to label if last CMP was equal (Zero Flag = 1)
jne label	Jump if not equal
jge label	Jump if greater than or equal
jle label	Jump if less than or equal
jmp label	Unconditional jump
inc reg	Add 1 to register
dec reg	Subtract 1 from register
loop label	Decrement ECX; jump if ECX $\neq$ 0
push reg	Save register value on top of stack
pop reg	Restore register value from top of stack
call proc	Jump to procedure and save return address on stack
ret	Return from procedure (pop return address from stack)
movzx dst, src	Move small value to larger register, zero-extend upper bits
OFFSET var	Get memory address of a variable
[reg]	Dereference: read value stored at the address in reg
BYTE PTR [reg]	Treat the memory location as a byte-sized value
DWORD PTR [reg]	Treat the memory location as a 4-byte (32-bit) value